REFERENCIA OFICIAL DE INGENIERÍA

Árbol de Estructura de Directorios y Archivos

Medicent Backend V3 — Guía de Nomenclatura Exacta para Cristian Beltrán y Compañero

PROYECTO	ARQUITECTURA	METODOLOGÍA
Backend_Medicent_v3	Modular por Capas (MVC + Services)	TDD & Pair Programming (XP)

> 🗺️ **El Mapa del Tesoro (Para entenderlo en un minuto)**
>
> Este mapa les dice a Cristian y a ti el **nombre exacto** de cada cajita y cada hojita que van a crear. Si una hojita se llama `usuario.py`, no deben llamarla `user.py` ni `usuarios.py`, sino exactamente como dice aquí para que todo encaje a la perfección como piezas de LEGO.

1. Árbol Completo de Directorios y Archivos

```
Backend_Medicent_v3/
│
├── app/                              # Núcleo modular de la aplicación
│   │
│   ├── config/                       # Configuraciones de entorno y base de datos
│   │   ├── __init__.py
│   │   └── settings.py               # Clase Config (MySQL) y TestConfig (SQLite memoria)
│   │
│   ├── database/                     # Inicialización y extensiones ORM
│   │   ├── __init__.py
│   │   └── connection.py             # Instancias de SQLAlchemy, Bcrypt, JWT y Migrate
│   │
│   ├── models/                       # Capa de entidades ORM en Español
│   │   ├── __init__.py
│   │   ├── usuario.py                # Modelo 'usuarios' (paciente, cuidador, admin)
│   │   ├── paciente_cuidador.py      # Tabla asociativa N:M 'pacientes_cuidadores'
│   │   ├── medicamento.py            # Modelo 'medicamentos' (catálogo farmacológico)
│   │   ├── tratamiento.py            # Modelo 'tratamientos' (dosis, frecuencia, fechas)
│   │   └── registro_toma.py          # Modelo 'registros_tomas' (seguimiento de tomas)
│   │
│   ├── services/                     # Capa de Lógica de Negocio y Algoritmos
│   │   ├── __init__.py
│   │   ├── auth_service.py           # Hashing de passwords, login JWT y asignaciones
│   │   ├── medicamento_service.py    # Validaciones y catálogo de medicamentos
│   │   ├── tratamiento_service.py    # Motor de calendarización automática de tomas
│   │   └── toma_service.py           # Registro de tomas (tomada, omitida, retrasada)
│   │
│   ├── controllers/                  # Controladores HTTP (Manejo de Request/Response)
│   │   ├── __init__.py
│   │   ├── auth_controller.py        # Manejo de endpoints /api/auth/*
│   │   ├── medicamento_controller.py # Manejo de endpoints /api/medicamentos/*
│   │   ├── tratamiento_controller.py # Manejo de endpoints /api/tratamientos/*
│   │   └── toma_controller.py        # Manejo de endpoints /api/tomas/*
│   │
│   ├── routes/                       # Definición de Blueprints y Endpoints REST
│   │   ├── __init__.py
│   │   ├── auth_routes.py            # Blueprint /api/auth/*
│   │   ├── medicamento_routes.py     # Blueprint /api/medicamentos/*
│   │   ├── tratamiento_routes.py     # Blueprint /api/tratamientos/*
│   │   └── toma_routes.py            # Blueprint /api/tomas/*
```

```
│   │
│   ├── middlewares/                    # Seguridad y Control de Acceso
│   │   ├── __init__.py
│   │   └── auth_middleware.py          # Decorador @rol_requerido para tokens JWT
│   │
│   └── __init__.py                     # Application Factory (create_app) con CORS
│
├── database/                           # Scripts SQL de Base de Datos MySQL
│   ├── schema.sql                      # Script DDL de tablas y restricciones (medicent_v3)
│   └── seed.sql                        # Datos de prueba iniciales
│
├── tests/                              # Suite de Pruebas Unitarias e Integración (TDD)
│   ├── __init__.py
│   ├── base_test.py                    # Configuración base con SQLite en memoria
│   ├── test_modelos.py                 # Pruebas unitarias de modelos y relaciones
│   ├── test_servicios.py               # Pruebas unitarias de lógica y cálculo de tomas
│   └── test_rutas_api.py               # Pruebas de integración HTTP de endpoints
│
├── documentos/                         # Documentación técnica y metodológica oficial
│   ├── Guia_Maestra_Desarrollo_Medicent_V3.pdf
│   ├── Metodologia_Pair_Programming_y_TDD.pdf
│   └── Arbol_Estructura_Archivos_Medicent_V3.pdf
│
├── DESARROLLO_EN_PAREJAS.md            # Metodología Pair Programming (Cristian y Tu Parte)
├── README.md                           # Guía técnica de instalación y ejecución
├── .env / .env.example                 # Parámetros de entorno
├── .gitignore                          # Filtro para control de versiones Git
├── requirements.txt                    # Dependencias oficiales del proyecto
└── run.py                              # Punto de arranque del servidor Flask
```

2. Tabla Detallada de Archivos y Responsabilidades

Ruta / Directorio	Nombre Exacto de Archivo	Responsable	Función en el Backend
app/config/	`__init__.py`	Ambos	Inicializador del paquete de configuración.
	`settings.py`	Ambos	Clases `Config` (MySQL para producción/desarrollo) y `TestConfig` (SQLite memoria).
app/database/	`__init__.py`	Ambos	Inicializador del paquete de base de datos.
	`connection.py`	Ambos	Instancias globales desacopladas: `db`, `bcrypt`, `jwt`, `migrate`.
app/models/	`__init__.py`	Ambos	Exporta todos los modelos para SQLAlchemy.
	`usuario.py`	Cristian	Modelo `Usuario` (id, nombre, correo, contrasena_hash, rol, creado_en).
	`paciente_cuidador.py`	Cristian	Tabla asociativa N:M `pacientes_cuidadores` (cuidador_id, paciente_id, creado_en).
	`medicamento.py`	Tu Parte	Modelo `Medicamento` (id, nombre, descripcion, forma_farmaceutica, creado_en).
	`tratamiento.py`	Tu Parte	Modelo `Tratamiento` (id, paciente_id, medicamento_id, dosis, frecuencia_horas, fechas).
	`registro_toma.py`	Tu Parte	Modelo `RegistroToma` (id, tratamiento_id, fecha_hora_programada, estado, notas).
app/services/	`__init__.py`	Ambos	Inicializador del paquete de lógica de negocio.
	`auth_service.py`	Cristian	Lógica de hashing Bcrypt, generación de tokens JWT y asignación de cuidadores.
	`medicamento_service.py`	Tu Parte	CRUD de catálogo de fármacos y validaciones farmacológicas.
	`tratamiento_service.py`	Tu Parte	**Motor de tomas:** cálculo matemático de intervalos según `frecuencia_horas`.
	`toma_service.py`	Tu Parte	Gestión de estados de tomas: transición a tomada, omitida o retrasada.
app/controllers/	`__init__.py`	Ambos	Inicializador de controladores.
	`auth_controller.py`	Cristian	Controlador de registro, login y vinculación paciente-cuidador.
	`medicamento_controller.py`	Tu Parte	Controlador de listado y creación de fármacos en catálogo.
	`tratamiento_controller.py`	Tu Parte	Controlador para crear prescripciones y consultar tratamientos de pacientes.
	`toma_controller.py`	Tu Parte	Controlador para marcar tomas y consultar calendario del paciente.
app/routes/	`__init__.py`	Ambos	Exporta y registra todos los Blueprints en la app.
	`auth_routes.py`	Cristian	Declaración de rutas `/api/auth/*`.
	`medicamento_routes.py`	Tu Parte	Declaración de rutas `/api/medicamentos/*`.
	`tratamiento_routes.py`	Tu Parte	Declaración de rutas `/api/tratamientos/*`.
	`toma_routes.py`	Tu Parte	Declaración de rutas `/api/tomas/*`.

Ruta / Directorio	Nombre Exacto de Archivo	Responsable	Función en el Backend
app/middlewares/	`__init__.py`	Ambos	Inicializador del paquete de middlewares.
	`auth_middleware.py`	Cristian	Decorador `@rol_requerido(*roles)` para verificar tokens JWT y claims de rol.
app/	`__init__.py`	Ambos	Application Factory: función `create_app(config_class)` con CORS.
database/	`schema.sql`	Ambos	Script DDL con creación de tablas y restricciones en MySQL (medicent_v3).
	`seed.sql`	Ambos	Datos de prueba iniciales (usuarios semilla, medicamentos y asignaciones).
tests/	`__init__.py`	Ambos	Inicializador de suite de pruebas TDD.
	`base_test.py`	Ambos	Clase base con `setUp` y `tearDown` sobre SQLite en memoria.
	`test_modelos.py`	Ambos	Pruebas de modelos: Usuario, PacienteCuidador, Medicamento, Tratamiento, Toma.
	`test_servicios.py`	Ambos	Pruebas del motor algorítmico de tomas, hashing y lógica de negocio.
	`test_rutas_api.py`	Ambos	Pruebas de integración HTTP completa sobre todos los endpoints REST.
Raíz /	`run.py`	Ambos	Punto de arranque del servidor Flask (`app.run()`).
	`requirements.txt`	Ambos	Lista de dependencias oficiales (Flask, SQLAlchemy, Bcrypt, PyJWT, CORS, etc.).
	`.env / .env.example`	Ambos	Variables de entorno (claves secretas JWT, credenciales de BD).
	`.gitignore`	Ambos	Filtro de exclusión de Git (ignorar `__pycache__`, `.env`, `venv/`).
	`README.md`	Ambos	Guía técnica de instalación y ejecución del proyecto.
	`DESARROLLO_EN_PAREJAS.md`	Ambos	Norma de pair programming, roles y flujo de Git.